Videre: Studying Realism in Videos

Anna Wu, Iris Xu, Ria Garg

December 5, 2025

1 Introduction

Recent advances in text-to-video generation models such as Sora, Pika, and ModelScope have enabled the creation of increasingly realistic AI videos with coherent and semantically plausible motion dynamics. It has thus become important to understand how close current systems are to achieving indistinguishably realistic video generation and to identify the subtle discrepancies between authentic and synthetic video that still limit realism in generated videos (Roca et al., 2025; Ramachandran et al., 2021; Heidari et al., 2024).

This work explores various machine learning techniques that leverage pretrained video encoders to determine whether a given video is real (originally recorded on a physical video-recording device) or AI-generated. In the latter case, some of the models are able to highlight areas of the video with unrealistic visuo-temporal artifacts and inconsistencies, such as flickering or unnatural motion, that may suggest a synthetic origin. These insights can both inform targeted improvements in video generation models and help automate the detection of generated videos in real-world contexts, particularly for videos derived from authentic footage but subsequently altered or tampered with, the identification of which may have significant implications in legal evidence verification, fraud prevention, and combating misinformation.

2 Related Work

We surveyed related work for datasets and/or possible inspiration for methods (Chang et al., 2024; Nguyen et al., 2023; Yan et al., 2023). Eventually, we discovered the DeeptraceReward (Fu et al., 2025) dataset, which provides a 4.3K detailed annotations across 3.3K high-quality generated videos, which serves as a benchmark for models trained to make a binary classification on whether a video is real or fake, as well as models trained for deepfake trace detection. These traces are defined as attributes that are the most significant in indicating whether or not a video is a deepfake.

While there has been much work on detecting deepfake videos that specifically include human faces, not as much exploration has been done on detecting general deepfakes, including videos whose foci are buildings, animals, natural landscapes, etc. Recent studies such as Ramachandran et al. (2021) have investigated deepfake detection methods such as FaceSwap, Face2face, FaceShifter; however, it remains challenging to generalize any of these "face-focused" methods to perform well on benchmarks like DeeptraceReward that include a diverse range of video content. Our project aimed to generalize our models to other domains by focusing on training models that are capable of identifying frame-by-frame patches that are indicative of fake videos in our classification tasks.

3 Dataset and Features

Our models are trained on the DeeptraceReward dataset described previously (Fu et al., 2025). We first converted the dataset into frame representations as numpy arrays. We then performed feature extraction on cropped versions of each frame using DinoV2. The videos were originally rectangles and were cropped during feature extraction to 518 x 518 (px) centered squares. The videos were then passed through the DinoV2 'ViT small patch14' model, which divides each video into a 37x37 grid of 14px by 14px patches and for each patch extracts a set of 384 patch-specific features. Finally, the data was split into train-test-val sets in a 70-15-15 ratio respectively. These split feature embeddings were then passed into our various models for training and testing.

For ablation studies, we also performed feature extraction using ResNet-50 to compare the performance of our models on different features. Please see Ablation Studies below for more information.

4 Methods

Our training pipeline follows a supervised-learning workflow built around pretrained DINOv2 video embeddings. Below, we describe the architecture of the models used and how we adapted each model to our use case.

For each model, we load the feature matrix X, label vector y, and a JSON file specifying train/val/test indices. The split file determines the partitions.

4.1 Logistic Regression

Our baseline classifier is binary logistic regression. Logistic regression is a generalized linear model used for binary classification. Given an input feature embedding vector $x \in \mathbb{R}^d$, the model computes a logit $t = w^T x + b = \theta^T x$, where $\theta \in \mathbb{R}^d$ includes both weights w and bias b which are learnable parameters. The model then maps the logit through a sigmoid function in order to ensure that the predictions are somewhere in between 0 and 1. The parameters are optimized using L2-regularized binary cross-entropy loss

$\mathcal{L}(w, b) = -\sum_{i=1}^{N} [y_i \log \sigma(t_i) + (1 - y_i) \log(1 - \sigma(t_i))] + \lambda \|w\|_2^2$. The final training model is serialized and stored as an artifact along with its metrics for deeper analysis. This model serves as a test of how linearly separable these embeddings are.

4.2 Trees: Random Forest and Decision Trees

A random forest consists of an ensemble of decision trees, each trained on a sample of the data and randomly selected subset of features. Each tree predicts a class label and majority voting is used to obtain the final random forest prediction.

A decision tree is a conceptually simple architecture where the model splits the data into two groups at every node of the tree based on a certain metric. Our model takes the feature vectors for each frame of a video as input, and passes each of these vectors into a decision tree. The classification of the video is then determined by averaging the probabilities that each frame's decision tree outputted.

4.3 Multilayer Perceptron Neural Network

A MLP neural network is a type of non-linear model/parameterization. In this case we have implemented a 2-layer MLP NN architecture consisting of two linear layers with ReLU (Rectified Linear Unit) activation functions applied after each, allowing the network to learn complex non-linear patterns in the data.

4.4 Convolutional Neural Network

We implemented a 2-D convolutional neural network (PatchCNN) that processes patch-level vision features for binary classification. The network takes as input a 3-D tensor of shape (embedding_dim, H, W) where embedding_dim is the number of feature channels (patch embedding dimension from the vision model) and H x W represents the spatial grid of image patches. The architecture consists of multiple convolutional layers with batch normalization and ReLU activation functions, followed by max pooling operations to progressively reduce spatial dimensions while extracting hierarchical features. The convolutional layers capture local spatial patterns across the patch grid, which allows the model to learn relationships between neighboring patches. After the convolutional blocks the spatial features are flattened and passed through fully-connected layers that aggregate the learned representations across the entire image. The final layer outputs logits for the two classes, which are converted to class probabilities using a softmax activation during inference. The model is trained using cross-entropy loss with Adam optimization and early stopping based on validation loss.

5 Experiments/Results/Discussion

We evaluate four model families – logistic regression, trees, neural networks, and convolutional neural networks – on the DINOv2 video embeddings. The neural networks are trained on the same 70/15/15 train-validation-test split, and the rest of the classification models are trained on a mini-dataset due to resource constraints. All are assessed using accuracy, precision, recall, F1 score, ROC-AUC, and PR-AUC. We also analyze confusion matrices and ROC/PR curves. These analyses allow us to compare the strengths and weaknesses of the model families we chose. We describe the graphs in detail at first and then for the following algorithms, we highlight any differences or improvements.

5.1 Logistic Regression

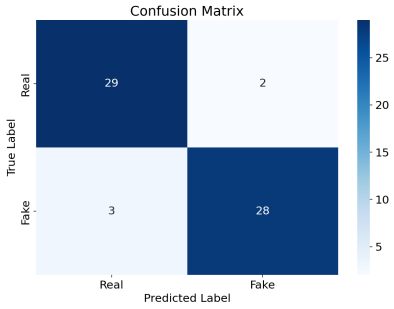
Logistic regression provides a natural baseline for evaluating whether the DINOv2 embedding space is linearly separable with respect to real vs. AI-generated videos. We trained the logistic regression model described above. The metrics for our training, evaluation, and testing are shown in Table 1.

Split	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	Loss
Training	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.0016
Evaluation	0.9677	0.9394	1.0000	0.9688	0.9948	0.9948	0.1306
Testing	0.9194	0.9333	0.9032	0.9180	0.9823	0.9837	0.1955

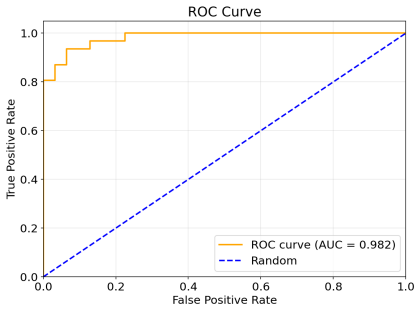
Table 1: Comparison of six metrics across training, evaluation, and testing splits.

The model achieves very high performance on the training set, as shown by the accuracy, precision, recall, and F1. This indicates that the linear decision boundary is sufficient for the small training subset and most likely overfit to the training data, but despite overfitting, the performance on the evaluation and test splits remains relatively high. This indicates that the the model generalized reasonably well and still captures meaningful structure in the data. The loss is also relatively small, though it is the smallest in training. Note that the recall in particular takes a large hit for testing (dropping almost 10%), which may indicate that the test set had more challenging fake videos. As shown in Figure 1a, the classifier produces three false negatives, which refers to the number of fake videos that were classified as real, and two false positives, which refers to the number of real videos classified as fake. Clearly, there is a strong diagonal here, with most of the samples concentrated in either the top left or bottom right. The number of false-negatives and false-positives is very low, but since $3 > 2$, the model is slightly conservative in the sense that it missed a few fake videos but missed even fewer real videos. There seem to be some insights in fake videos that a linear classifier cannot easily separate.

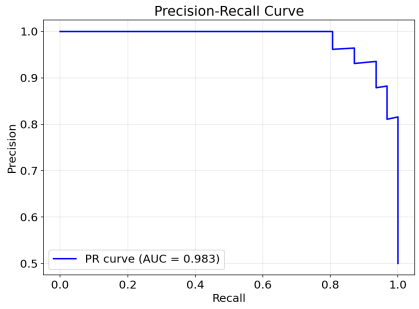
Ideally, the curve in Figure 1b should hug the top-left corner, which would indicate high True Positive Rate (TPR) with low False Positive Rate (FPR). The shape of the graph indicates that the model achieved high recall and low false positives. Overall, the model separates classes pretty well across the thresholds. Overall, there is strong separability between the two classes. The PR curve in Figure 1c also



(a) Confusion matrix for logistic regression model.



(b) ROC curve for logistic regression model.



(c) Precision–Recall curve for logistic regression model.

Figure 1: Performance evaluation plots for the logistic regression model.

reflects high precision-recall balance. As recall increases (at least until 0.8), the precision does not drop. The fact that the PR-AUC score is higher than the ROC-AUC score implies that the model performs well on the positive class, which is the class of fake videos.

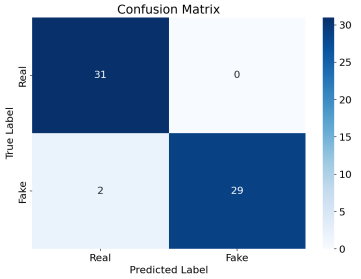
Overall, logistic regression demonstrates that the DINOv2 features space is sufficiently expressive that even linear boundaries can achieve good classification accuracy. However, the model's limited expressiveness prevents it from capturing nuanced, nonlinear inconsistencies that appear in more challenging generative videos.

5.2 Random Forest and Decision Tree

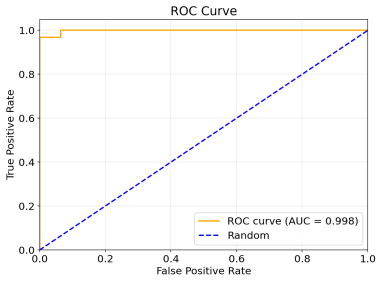
Random forests represent a non-linear alternative that can potentially capture feature interactions missed by a linear model. As shown

Split	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Training	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Evaluation	0.9355	0.9655	0.9032	0.9333	0.9896	0.9884
Testing	0.9677	1.0000	0.9355	0.9667	0.9979	0.9980

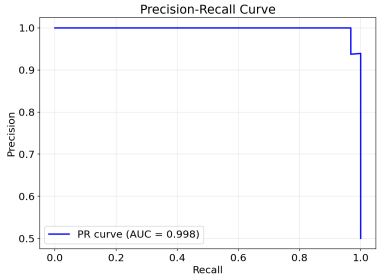
Table 2: Comparison of six metrics across training, evaluation, and testing splits.



(a) Confusion matrix for random forest model.



(b) ROC curve for random forest model.



(c) Precision–Recall curve for random forest model.

Figure 2: Performance evaluation plots for the random forest model.

in Table 2, the random forest outperforms the logistic regression model in testing across nearly all metrics. Figure 2a shows zero false positives, which means that no real videos were misclassified as fake, and two false negatives. These statistics collectively are better than logistic regression. The superior performance of random forests likely arises from their ability to model nonlinear interactions - DINOv2 embeddings are semantically rich, and subtle physical differences likely manifest through nonlinear relationships between features.

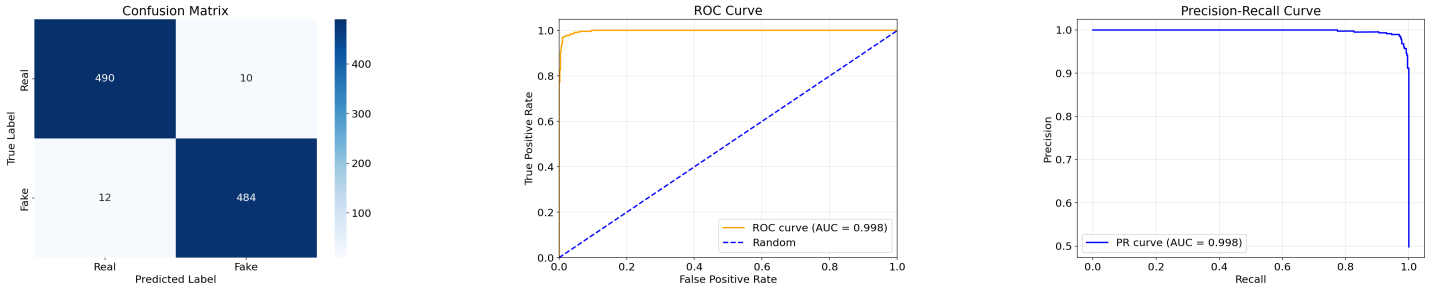
As a tangential further exploration, we tried a decision tree model with a depth of 8 which takes in per-frame feature embeddings for each video, and passes each of these feature embeddings into the same decision tree. The probability outputs of these trees are classified into either "real" or "fake" based on a 0.5 threshold. A majority vote is then taken across the frames to determine whether the video is real or fake. The accuracy on the validation set using ResNet-50 feature extractions was .7705, and the accuracy achieved on the test set was .5781.

5.3 MLP Neural Network (NN)

We also implemented a simple MLP neural network with ReLU. The neural network used train/validate/test (70/15/15) data partitioned from the full DeeptraceReward dataset.

Split	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Training	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Evaluation	0.9698	0.9668	0.9708	0.9688	0.9965	0.9961
Testing	0.9779	0.9798	0.9758	0.9778	0.9978	0.9977

Table 3: Comparison of six metrics across training, evaluation, and testing splits.



(a) Confusion matrix for the NN. (b) ROC curve for the NN. (c) Precision–Recall curve for the NN.

Figure 3: Performance evaluation plots for the MLP neural network.

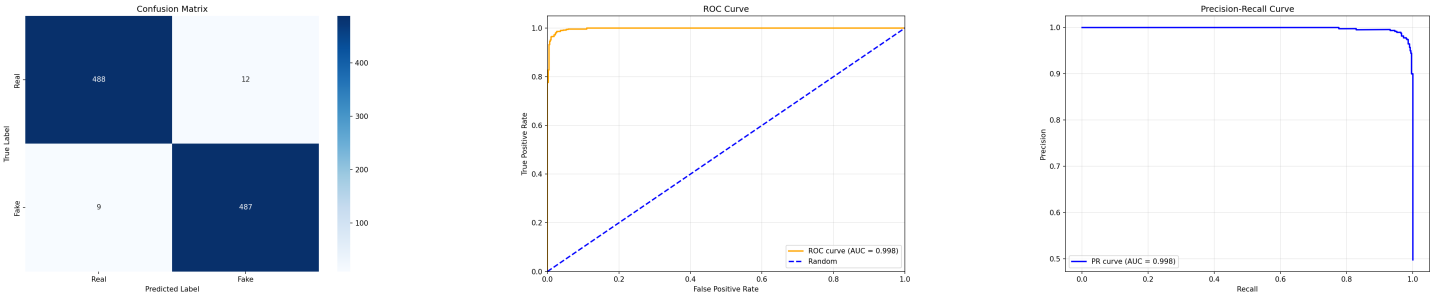
As demonstrated in Table 3, the MLP NN narrowly outperformed both the random forest and logistic regression classifiers in accuracy. We noticed that there was a larger increase in accuracy from logistic regression to random forest than there was from random forest to MLP, largely because the accuracies are in the higher range. This also might be because the neural network architecture is not as helpful for the DINOv2 video embeddings.

5.4 Convolutional Neural Network (CNN)

While the previous models operate on a single DINOv2 embedding per video, our convolutional neural network (CNN) operates on patch-token grids derived from DINOv2. For each video, we obtain a grid of patch embeddings of shape (H, W, D), where each of the $H \times W$ spatial positions has a D-dimensional DINOv2 feature vector. Shown below are the test metrics.

Split	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Testing	0.9789	0.9760	0.9819	0.9789	0.9979	0.9978

Table 4: Six metrics for testing split.



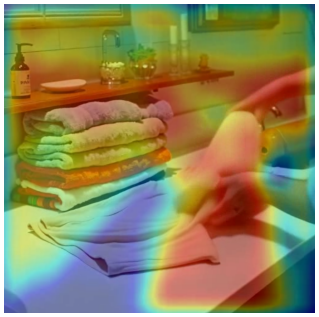
(a) Confusion matrix for the CNN. (b) ROC curve for the CNN. (c) Precision–Recall curve for the CNN.

Figure 4: Performance evaluation plots for the CNN.

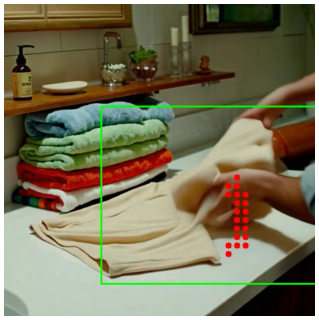
We also implemented GradCAM (Selvaraju et al., 2016) on top of our trained CNN to generate heatmaps and visually identify regions of interest for videos identified as fake. The heatmap script can be invoked to color a given video according to the contributions of each regional patch to its label, where redder regions of the video were more influential in determining its "fakeness" and bluer regions had negligible effect on the classification. We also provide a function for highlighting the centers of the patches which were most influential.

We evaluated GradCAM with the CNN model on 300 fake videos with an accuracy of 0.54. We measured accuracy by whether any of the centers of the top 20 "fakest" patches, which usually clustered near to each other, fell inside of the bounding box provided by the DeeptraceReward dataset surrounding the area of the video containing fake artifacts. We attribute this somewhat unimpressive performance to several qualitative observations. First, the fake-looking objects may move around throughout the video, but the GradCAM heatmaps and top patches of interest we generated are static because the features we used to train the CNN were averaged throughout the video. Furthermore, the bboxes were created by human annotators and are meant to generally, not strictly, indicate the area with fake

artifacts; sometimes, artifacts do not cleanly fit inside a bbox, or move in and out of its borders. Finally, many videos had multiple fake features, and we sometimes saw GradCAM emphasize a second fake artifact which was not the artifact targeted by the video's bbox.



(a) Heatmap for an example video.



(b) Points of interest for an example video in red. DeeptraceReward bbox in green.

Figure 5: In this frame from a video which was correctly classified as fake, the subject's hands are not anatomically accurate, and the items on the bathroom counter are unusual (e.g. disconnected faucet head). The heatmap identifies these regions as contributing most significantly to the CNN's "fake" decision, and the top 20 most influential patches all cluster around the suspicious hand.

6 Ablation Studies with ResNet

6.0.1 Feature Extraction

Because DinoV2 is known to have highly rich and effective features for video classification, we tested another feature extraction algorithm to determine how the effect of our features vs. our model on the prediction accuracy.

In terms of runtime, DinoV2 had a runtime of between six and seven hours for feature extraction for 400 videos, whereas Resnet-50 had a runtime of about three minutes using the same script structure, despite the fact that DinoV2 produces a feature vector of length 384, and Resnet-50 produces a feature vector of length 2048. This suggests that the DinoV2 features are much more difficult to extract and represent richer and/or more significant information. We therefore expected all of our models to perform more poorly on the Resnet-50 features than the DinoV2 features. We re-ran our classification models — logistic regression, random forest, and decision tree — on the features extracted using Resnet-50, and obtained the following testing results:

Algorithm	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.4839	0.4444	0.1290	0.2000	0.4651	0.4681
Random Forest	0.4032	0.3333	0.1935	0.2449	0.4537	0.4564
Decision Tree	0.9048	0.8824	0.9375	0.9091	0.9556	0.9626

Table 5: Comparison of six metrics across three algorithms.

Across all algorithms, performance on ResNet features is close to random, with ROC-AUC values near 0.45–0.46 and F1 scores below 0.25, and thus substantially lower than the training with the DINOv2 embeddings. This suggests that the two classes are nearly indistinguishable in the ResNet embedding space. This might be because ResNet-50 is trained on ImageNet and is optimized for natural-image semantics whereas modern video generation models are exceptionally strong at producing locally plausible textures and edges, making these low-level features poor indicators of synthetic content.

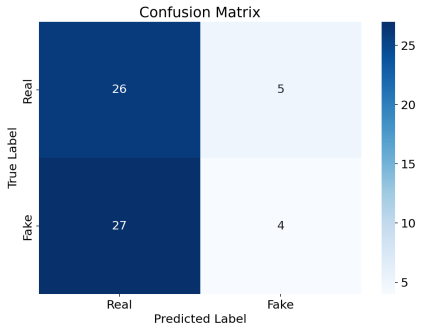
7 Conclusion and Future Work

In this project, we investigated how pretrained video encoders can be used to distinguish real videos from AI-generated ones. We evaluated a spectrum of models, from logistic regression to convolutional neural networks. The linear models achieved high accuracy and AUC, indicating that the DINOv2 feature space does a good job of separating real and synthetic content, and non-linear models further closed the gap. We then analyzed the DINOv2 patch tokens in their original spatial layout via a CNN, which achieved the strongest test performance while also providing spatially meaningful explanations through Grad-CAM. Our bounding-box analysis on the heatmaps produced showed that the regions that were most influential in the classification often overlapped substantially with those regions where human annotators agree that generative models break.

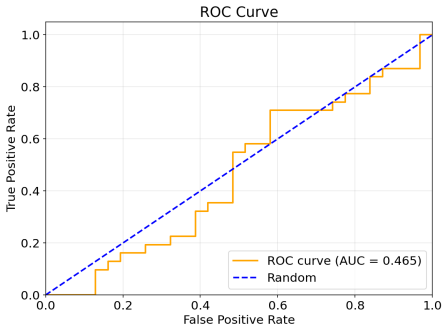
Ablation experiments with ResNet-50 features underscored that not all pretrained representations can perform strongly. While ResNet-based features led to much faster extraction, they consistently underperformed DINOv2 features in downstream classification. In terms of future work, our current models treat time only implicitly (either by pooling over frames or aggregating per-frame decisions). A natural next step is to incorporate explicit temporal modeling through 3D CNNs in order to capture artifacts such as inconsistent motion, temporal aliasing, or abrupt scene dynamics. Second, our localization pipeline is based on Grad-CAM applied post hoc to a classifier trained only on video-level labels. Training models directly for spatiotemporal segmentation or bounding-box prediction, using the DeeptraceReward annotations as supervision, could further improve localization quality and enable more fine-grained explanations.

8 Appendices

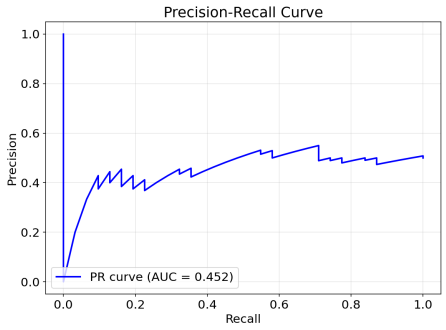
8.1 Ablation Studies - ResNet



(a) Confusion matrix for logistic regression model.



(b) ROC curve for logistic regression model.



(c) Precision–Recall curve for logistic regression model.

Figure 6: Performance evaluation plots for the logistic regression model using ResNet embeddings.

9 Contributions

- Anna Wu: GCP VM setup and management, MLP NN and CNN architecture and implementation.
- Iris Xu: data research and pre-processing, feature embeddings extraction, decision tree architecture and implementation.
- Ria Garg: logistic regression and random forest architecture and implementation, metric processing and results/ablation analysis.

All contributed to ideating, work sessions, and writing this final report.

References

Chirui Chang, Zhengzhe Liu, Xiaoyang Lyu, and Xiaojuan Qi. 2024. What Matters in Detecting AI-Generated Videos like Sora?

Xingyu Fu, Siyi Liu, Yinuo Xu, Pan Lu, Guangqiuse Hu, Tianbo Yang, Taran Anantasagar, Christopher Shen, Yikai Mao, Yuanzhe Liu, Keyush Shah, Chung Un Lee, Yejin Choi, James Zou, Dan Roth, and Chris Callison-Burch. 2025. DeeptraceReward. https://deeptracereward.github.io/

Arash Heidari, Nima Jafari Navimipour, Hasan Dag, and Mehmet Unal. 2024. Deepfake detection using deep learning methods: A systematic and comprehensive review. e1520 pages. arXiv:https://wires.onlinelibrary.wiley.com/doi/pdf/10.1002/widm.1520 doi:10.1002/widm.1520

Tai D. Nguyen, Shengbang Fang, and Matthew C. Stamm. 2023. VideoFACT: Detecting Video Forgeries Using Attention, Scene Context, and Forensic Traces. arXiv:2211.15775 [cs.CV] https://arxiv.org/abs/2211.15775

Sreeraj Ramachandran, Aakash Varma Nadimpalli, and Ajita Rattani. 2021. An Experimental Evaluation on Deepfake Detection using Deep Face Recognition. 6 pages. doi:10.1109/ICCST49569.2021.9717407

Thomas Roca, Anthony Cintron Roman, Jehú Torres Vega, Marcelo Duarte, Pengce Wang, Kevin White, Amit Misra, and Juan Lavista Ferres. 2025. How good are humans at detecting AI-generated images? Learnings from an experiment. arXiv:2507.18640 [cs.HC] https://arxiv.org/abs/2507.18640

Ramprasaath R. Selvaraju, Abhishek Das, Ramakrishna Vedantam, Michael Cogswell, Devi Parikh, and Dhruv Batra. 2016. Grad-CAM: Why did you say that? Visual Explanations from Deep Networks via Gradient-based Localization. arXiv:1610.02391 http://arxiv.org/abs/1610.02391

Zhiyuan Yan, Yong Zhang, Xinhang Yuan, Siwei Lyu, and Baoyuan Wu. 2023. DeepfakeBench: A Comprehensive Benchmark of Deepfake Detection. 4534–4565 pages. https://proceedings.neurips.cc/paper_files/paper/2023/file/0e735e4b4f07de483cbe250130992726-Paper-Datasets_and_Benchmarks.pdf

Algorithm	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Logistic Regression	0.9194	0.9333	0.9032	0.9180	0.9823	0.9837
Random Forest	0.9677	1.0000	0.9355	0.9667	0.9979	0.9980
Decision Tree	0.8413	0.7750	0.9688	0.8611	0.9607	0.9728
NN	0.9779	0.9798	0.9758	0.9778	0.9978	0.9977
CNN	0.9789	0.9760	0.9819	0.9789	0.9979	0.9978

Table 6: Comparison of six metrics across three algorithms on test sets, using DINOv2 features.

Algorithm	Test Accuracy	Training Accuracy	Train Size	Test Size	Precision	Recall	F1	ROC-AUC	Log Loss
Logistic Regression	0.9194	1.0000	292	62	0.9333	0.9032	0.9180	0.9823	0.1955
Random Forest	0.9677	1.0000	292	62	1.0000	0.9355	0.9667	0.9979	0.3951
Decision Tree	0.8413	0.5414	292	62	0.7750	0.9688	0.8611	0.9607	0.3547
NN	0.9779	1.0000	4646	996	0.9798	0.9758	0.9778	0.9978	0.0974
CNN	0.9789	.9998	4646	996	0.9760	0.9819	0.9789	0.9979	0.1592

Table 7: Comparison of six metrics across three algorithms on test sets, using DINOv2 features.